CSharp gyakorlófeladatok az első öt héthez

Információábrázolás és algoritmizálás alapjai

Hallgatói feladatlap kezdő konzolos programokhoz

Öt kisebb programon dolgozol. Előbb számold ki vagy tippeld meg az eredményt, majd egészítsd ki az induló kódot, futtasd a programot, és magyarázd meg az eltéréseket. A kódvázban a TODO jelölés mutatja a saját feladatodat.

Hét	Gyakorlati cél	Becsült idő
1	Beolvasás, változók és alapműveletek	35 perc
2	Átváltás négy számrendszer között	50 perc
3	Számjegyenkénti műveletek és komplemensek	65 perc
4	Bitműveletek és egész számábrázolás	45 perc
5	Lebegőpontos megfigyelések és kerekítés	55 perc

Indulás Visual Studióban

- Csomagold ki a ZIP teljes tartalmát egy ékezet nélküli útvonalra, például C:\CSharpMaraton alá.
- Nyisd meg az Indulo mappában az Indulo.sln fájlt. A kiválasztott Het1–Het5 projektre jobb gombbal kattintva állítsd be indítási projektként.
- Nyisd meg a Program.cs fájlt. Ctrl+F5 indítja a programot. Hibakereséshez tegyél töréspontot a bal margóra, és használd az F5, majd F10 billentyűt.

A mellékelt projektek .NET 9-re készültek. Ha más keretrendszerű konzolprojektben dolgozol, az oktatóval egyeztetett projektbe másold át az adott Program.cs tartalmát.

Munkamód

A TODO sorok kezdetben szándékosan hiányos számításokat tartalmaznak, ezért több kimenet hibás lesz. Haladj feladatszámonként. A KÉSZ SEGÉDRÉSZ a beolvasást ellenőrzi; ezt most nem kell újraírnod. A forráskódokban a tizedesjel pont, a kimenet is pontot használ.

Önellenőrzéskor három dolgot nézz: lefut-e a program, egyezik-e a kapott eredmény a mintával, és el tudod-e mondani, mit jelent a megírt kifejezés.

Rövid kódolási segédlap

A feladatokban az alábbi elemek fordulnak elő. Az új sorokat először az oktatóval együtt nézd meg; a készen adott részeket nem kell kívülről megtanulnod.

Elem	Jelentés
int	Egész szám, például darabszám.
double	Bináris lebegőpontos szám.
string és char	Szöveg és egyetlen karakter.
= és ==	Értékadás és egyenlőségvizsgálat.
/ és %	Osztás és az egész osztás maradéka.
if és else	A feltétel alapján más utasítást hajtunk végre.
while	Amíg a feltétel igaz, ismételjük a törzset.
for	Számlálóval vezérelt ismétlés.
foreach	Egy szöveg karakterein is végig tud lépkedni.
return	A függvény visszaad egy eredményt.

Számjegyek kezelése szövegként

```
string jegyek = "0123456789ABCDEF";
char betu = jegyek[13];       // D
int ertek = jegyek.IndexOf('D'); // 13
```

A szögletes zárójellel egy karaktert kérsz le; a sorszámozás nullától indul. A karakter + szöveg kifejezés a karaktert a szöveg elejére teszi. Ez a maradékos átváltásnál hasznos, mert fordított sorrendben keletkeznek a számjegyek.

Kész részprogram meghívása

```
string binaris = Tizesbol(13, 2);
```

A Tizesbol itt egy névvel ellátott részprogram. A 13-at és a 2-es alapot kapja bemenetként, majd egy szöveget ad vissza. A függvények szerkezetét készen kapod; a kijelölt számításokat egészíted ki bennük.

1 hét Alapvető konzolos műveletek

Program: Het1. Cél: adatokat kérsz be, megfelelő típusú változókban tárolod őket, majd számolsz velük. Az első héten csak a megadott tartományon belüli, helyes egész adatokat írd be.

1 1 Fejlesztői környezet és órai követelmények

- Jegyezd fel az oktatótól kapott hivatalos követelményeket, határidőket és értékelési szempontokat. A feladatlap önellenőrzési szempontjai ezekhez adnak gyakorlási segítséget.
- Hozz létre próbaként egy C# Console App projektet, és írass ki egy köszönést. Utána dolgozz a mellékelt Het1 projektben.
- Tegyél töréspontot egy számítás sorára. Lépj át rajta F10-zel, és figyeld meg a változó értékét a Locals vagy Watch ablakban.

1 2 A maratoni óra készletkalkulátora

A program a résztvevők nyomtatási igényét és a szükséges tea mennyiségét számolja. A beolvasás és a kiírás már a kódban van; a TODO 1.2 sorokat egészítsd ki.

Adat	Típus és tartomány
Név	string
Résztvevők száma	int, 1 és 200 között
Oldalak fejenként	int, 1 és 100 között
Egy oldal ára	int, 1 és 1000 forint között

- Számítsd ki az összes nyomtatott oldal számát.
- Számítsd ki a teljes nyomtatási költséget forintban.
- Fejenként 0,25 liter teával számolva határozd meg a szükséges litert. A kódban 0.25 szerepeljen, az eredmény típusa double legyen.

Gondold végig: a teának miért hasznos eltérő típust választani, mint a résztvevők számának?

1 hét Osztás és önellenőrzés

1 3 Egész és lebegőpontos osztás

Még futtatás előtt írd le a három kifejezés eredményét. Ezután ellenőrizd a programmal.

```
7 / 2
7 / 2.0
7 % 2
```

- Mondd el, miért különbözik az első két eredmény.
- A % nem százalékszámítás. Magyarázd el, mit ad vissza ebben a példában.

1 4 Percek átváltása

A TODO 1.4 sorokban számold át a 135 perces időtartamot teljes órákra és megmaradó percekre. Egész osztást és maradékképzést használj.

Ellenőrző adatok

Bemenet	Elvárt eredmény
20 fő, 2 oldal, 15 Ft	40 oldal; 600 Ft; 5 liter tea
3 fő, 5 oldal, 12 Ft	15 oldal; 180 Ft; 0.75 liter tea
135 perc	2 óra 15 perc

Választható bővítés

- Kérd be a percszámot is, és próbáld ki 45, 60, 125 és 180 perccel.
- A teljes költséget oszd fel egy külön bekért csapatszám között. Az osztás double eredményt adjon. Pozitív csapatszámmal dolgozz.

Mit tudsz a blokk végén

Beolvasol egy nevet és egész számokat, kiírsz szöveget és számítási eredményt, használod a *, / és % operátorokat, és hibakeresés közben megfigyeled a változókat.

2 hét Számrendszerek közötti átváltás

Program: Het2. Csak nem negatív, tízes értékük szerint 0 és 255 közötti egész számokkal dolgozunk. Az alap 2, 8, 10 vagy 16 lehet. Előjelet és tört részt most nem kezelünk.

2 1 Kézi előkészítés

- Írd fel a 45-öt kettes, nyolcas és tizenhatos alapban. A bináris felíráshoz használd a 32, 16, 8, 4, 2, 1 helyiértékeket.
- A 2D hexadecimális számot bontsd helyiértékes alakra. A D számjegy értéke 13.

2 2 Tízesből másik alapba

A Tizesbol függvény az ismételt egész osztás algoritmusát valósítja meg. Egészítsd ki a TODO 2.2 sort az osztási maradék kiszámításával.

- A maradék az aktuális számjegy értéke. A jegyek[maradek] ebből karaktert készít, például 13-ból D-t.
- Az új karakter az eredmény elejére kerül. Ezután a számot az alappal osztva folytatjuk.
- Külön figyeld meg a nulla bemenetet: ilyenkor a függvény közvetlenül a "0" szöveget adja vissza.

Az első ciklust közösen kövesd végig töréspontt al. A 45 kettes alapra váltásának első három állapotát írd le.

Osztandó	Egész hányados	Maradék
45	22	1
22	11	0
11	5	1

Kérdés: miért az utolsó maradék lesz a szám bal szélső számjegye?

2 hét Visszaalakítás és ellenőrzés

2 3 Helyiértékek feldolgozása balról jobbra

A Tizesbe függvényben a következő szabályt használd: az eddigi értéket megszorzod az alappal, majd hozzáadod az új számjegyet. Javítsd a TODO 2.3 sort.

```
ujErtek = regiErtek * alap + szamjegy;
```

A 2D feldolgozásakor az első lépés 0 × 16 + 2, a második 2 × 16 + 13. Figyeld meg az eredmeny változót mindkét lépés után.

2 4 Átváltás két választott alap között

A TODO 2.4 sorban először a forrásalakból kiszámított tízes értéket alakítsd a célalapra. A két saját függvényt használd. A Convert.ToInt32 és Convert.ToString kész konverzióit ennél a feladatnál csak utólagos ellenőrzésre használd.

Ellenőrző adatok

Tízes érték	Bináris	Oktális	Hexadecimális
0	0	0	0
13	1101	15	D
45	101101	55	2D
255	11111111	377	FF

Két alap közötti próba: 2D a 16-os alapban, célalap 2 → 101101. A kisbetűs 2d bemenetet a kész beolvasó is elfogadja.

Választható bővítés

- A Tizesbol függvény ciklusában írasd ki minden lépés előtt a számot, a hányadost és a maradékot.
- Próbálj érvénytelen számjegyet, például 102-t kettes alapban. Figyeld meg, hogyan kér új adatot a kész beolvasó.

3 hét Műveletek számjegyenként

Program: Het3. Az alap ismét 2, 8, 10 vagy 16. Az első operandus tízes értéke 0–255, a másodiké legfeljebb 15, és nem nagyobb az elsőnél. Így a kivonás eredménye nem negatív, a szorzás rövid marad.

A főprogram és a beolvasás kész. Az Osszead, Kivon és Szoroz függvényben csak a kijelölt sorokon dolgozz. Ezekben az algoritmusokban a teljes számszöveget ne alakítsd át tízes számmá a művelet elvégzéséhez. A kiírt tízes művelet ellenőrzésre szolgál.

3 1 Összeadás átvitellel

Először papíron add össze: 1101 és 0101 kettes alapban. Utána egészítsd ki az Osszead függvény három TODO sorát.

- Az aktualis legyen az első számjegy, a második számjegy és a korábbi átvitel összege.
- Az új eredményjegy az aktualis osztási maradéka az alap szerint.
- A következő átvitel az aktualis egész hányadosa az alappal.

A ciklus jobbról balra halad. Ha az egyik szöveg elfogyott, annak hiányzó számjegyét nullának vesszük. A végső átvitelt is ki kell írni.

3 2 Kivonás kölcsönzéssel

A Kivon függvény TODO 3.2 sorában az első jegyből vond ki a második jegyet és a korábbi kölcsönt. A negatív különbség kezelését a kód már tartalmazza.

- Ha a különbség negatív, adj hozzá egy alapnyi értéket, és a következő helyiértékhez jegyezz fel 1 kölcsönt.
- Ha nincs szükség kölcsönre, a következő kölcsön értéke 0 legyen.

Kövesd végig az 1000 − 0001 bináris kivonást. Figyeld meg, hogyan megy tovább a kölcsön több helyiértéken.

3 hét Szorzás és komplemensek

3 3 Szorzás ismételt összeadással

A Szoroz függvény eredménye kezdetben "0". A második operandus tízes értékének megfelelő számú ismétlésben add hozzá az első operandust a már megírt Osszead függvénnyel. Egészítsd ki a TODO 3.3 sort.

Ez egyszerű, kis számokra szánt szorzóalgoritmus. Az összeadások a választott alap számjegyein történnek; a ciklusszámláló csak az ismétléseket számolja. A 0-val szorzás is legyen helyes.

3 4 Egyes és kettes komplemens nyolc biten

- A program külön bekér egy 1–127 közötti pozitív számot, és nyolcbites bináris mintát készít róla.
- A TODO 3.4a részben minden 0-ból 1-et, minden 1-ből 0-t készíts. Ez az egyes komplemens.
- A TODO 3.4b sorban adj hozzá binárisan 1-et az egyes komplemenshez. Ez a kettes komplemensű negatív érték mintája.

A komplemens mindig adott bitszélességhez tartozik. Itt a 8 bit rögzített; most nem negatív bemenet szöveges átalakítását írjuk meg.

Alap és bemenet	Összeg	Különbség	Szorzat
2 1101 és 101	10010	1000	1000001
8 7 és 3	12	4	25
16 A és 3	D	7	1E
2 0 és 0	0	0	0

Komplemenspróba: +5 → 00000101; egyes komplemens → 11111010; kettes komplemens → 11111011, amely előjelesen −5.

Választható bővítés

Minden összeadási lépésben írasd ki a két számjegyet, a bejövő átvitelt, az eredményjegyet és a kimenő átvitelt. A táblázatot vesd össze a papíron végzett művelettel.

4 hét Bitek és előjeles értelmezés

Program: Het4. Egy byte típusú értékkel dolgozol. A biteket jobbról számozzuk: a legkisebb helyiérték sorszáma 0, a legnagyobbé 7.

4 1 Egy kiválasztott bit kezelése

A kezdő próba értéke 5, a vizsgált bit sorszáma 1. Az 5 nyolcbites alakja 00000101. Először papíron döntsd el, a kiválasztott bit most 0 vagy 1.

Művelet	Használandó gondolat
Maszk készítése	Az 1 balra tolása a bit sorszámával.
Bit ellenőrzése	& maszkkal, majd összehasonlítás nullával.
Bit bekapcsolása	| a maszkkal.
Bit kikapcsolása	& a ~ jellel megfordított maszkkal.
Bit átbillentése	^ a maszkkal.
Balra tolás egy hellyel	<< 1, majd az alsó 8 bit megtartása.

Egészítsd ki a TODO 4.1a–4.1f sorokat. Minden változat az eredeti byte értékből induljon; a műveleteket most ne egymás eredményén végezd.

A byte operandusok bitműveletei itt int eredményt adnak, ezért a visszatárolásnál (byte) konverzióra van szükség. A balra tolásnál az explicit unchecked megmutatja, hogy a kilépő felső bitet elhagyjuk.

4 2 Ugyanaz a nyolc bit kétféle értelmezéssel

A TODO 4.2 sorban unchecked sbyte konverzióval értelmezd ugyanazt a mintát előjeles nyolcbites értékként. Próbáld ki 127, 128 és 255 bemenettel.

Magyarázd meg: a 11111111 minta mikor jelent 255-öt, és mikor −1-et?

4 hét Adatméret és túlcsordulás

4 3 Bájtok és többszöröseik

A program bekéri egy fájl méretét bájtban. A TODO 4.3 sorokban számold ki a bitek számát, a kB-ban és a KiB-ban megadott méretet.

- 1 bájt = 8 bit; 1 kB = 1000 bájt; 1 KiB = 1024 bájt.
- A kB és KiB érték kiszámításánál 1000.0-val, illetve 1024.0-val ossz, hogy ne egész osztás történjen.

4 4 Túlcsordulás megfigyelése

A kész programban egy byte változó értéke 250, ehhez 10-et adunk. A kezdeti összeadás int eredményt ad. A TODO 4.4a sorban az int eredményt unchecked konverzióval tedd vissza byte-ba.

- Jósolj: az int eredmény, a visszatárolt byte és a checked konverzió eredménye azonos lesz?
- Futtasd a kész try/catch bemutatót. Ezt a hibakezelési szerkezetet most elég megfigyelni.
- A TODO 4.4b sorban a 127 + 1 összeget explicit unchecked sbyte konverzióval csonkold nyolc bitre.

Próba	Elvárt megfigyelés
5 érték, 1-es bit	Bekapcsolás 7; kikapcsolás 5; átbillentés 7; balra tolás 10
255 előjelesen	−1
4096 bájt	32768 bit; 4.096 kB; 4 KiB
250 + 10	int 260; unchecked byte 4; checked kivétel
127 + 1 sbyte-ba csonkolva	−128

Választható bővítés

Állítsd a bemenetet 128-ra, és figyeld meg a balra tolást. Miért 0 marad a nyolcbites eredmény, miközben az int típusban kiszámolt 128 << 1 értéke 256?

5 hét Lebegőpontos kísérletek

Program: Het5. Minden kísérlet előtt tippeld meg a kimenetet. Az eredményt és egy egymondatos magyarázatot is jegyezz fel. A kimenet tizedesjele pont.

5 1 Ugyanaz az összeg három típussal

A TODO 5.1 sorokban számold ki a 0,1 + 0,2 összeget float, double és decimal típussal. Figyeld meg az f és m utótagot. A kész kiírás több számjegyet mutat, és összehasonlítja az eredményt a megfelelő 0,3 értékkel.

- Írd le mindhárom kiírt összeget és az egyenlőségvizsgálat eredményét.
- Ha egy összehasonlítás igaz, abból következik-e, hogy a matematikai 0,1 pontosan tárolódott? A megjelenített számjegyeket is vizsgáld meg.

5 2 Normalizálás kettes alappal

A program pozitív számot kér be. Az 1 ≤ szignifikandusz < 2 feltételig felezi vagy duplázza az értéket. A TODO 5.2 sorokban vezesd a kitevőt.

- Felezéskor a kitevőt növeld 1-gyel; duplázáskor csökkentsd 1-gyel. A szorzat értékének meg kell maradnia.
- Próbák: 13.25 → 1.65625 × 2^3; 0.125 → 1 × 2^-3; 1 → 1 × 2^0.

A szignifikanduszt itt tízes alakban íratjuk ki. Az 1.65625 binárisan 1.10101. A program a tárolt double értéket normalizálja; nem fejti vissza kézzel az IEEE mezőket. A kiírt 2^3 matematikai jelölés. C#-ban a ^ kizáró VAGY; hatványozáshoz Math.Pow(2, kitevo) kell.

5 3 Kerekítés két szabállyal

A KerekitestMutat függvény TODO 5.3 soraiban hívd meg a Math.Round műveletet ToEven és AwayFromZero móddal. A bemenetek 2.5m, 3.5m, −2.5m és −3.5m; a decimal itt pontos félutas példát ad.

Hasonlítsd össze a kapott eredményeket. A ToEven félúton a páros szomszédot választja, az AwayFromZero félúton a nagyobb abszolútértékűt.

5 hét Pontosság és speciális értékek

5 4 Elvesző kis különbségek

- Futtasd a kész 16 777 216f + 1f példát. Ez a szám a float tartományába esik, mégis eltűnhet a hozzáadott 1. Túlcsordulás történt, vagy kerekítés?
- Az a = 1e16, b = −1e16, c = 1 értékekkel hasonlítsd össze az (a + b) + c és a + (b + c) eredményét.
- Nézd meg a double összeg és 0.3 abszolút eltérését. Miért nem elég csak két tizedesjegyet megjeleníteni a pontosság megítéléséhez?

5 5 Nagyon nagy és nagyon kis értékek

A következő sorok kész megfigyelési kísérletek. Írd le, mit jelez a három igazságérték, és keresd meg a kódban a hozzájuk tartozó számítást.

Kísérlet	Mit vizsgálsz
double.MaxValue × 2	Véges tartomány túllépése.
double.Epsilon / 2	A legkisebb pozitív double felének kerekítése.
0.0 / 0.0	NaN, azaz nem számérték keletkezése.

Fontos: double.Epsilon a legkisebb pozitív double érték. Nem általános hibahatár, és nem az 1 közelében mérhető pontosság. A NaN vizsgálatához a double.IsNaN metódust használd.

Választható bővítés

- A program végén egy Half értéket floatként írunk ki. Hasonlítsd össze a 0.1 közelítését a float és double eredményeivel.
- Írasd ki ugyanazt a double eredményt G17 és F2 formázással. Megváltozott-e a tárolt érték, vagy csak a megjelenített szöveg?

Záró önellenőrzés

Válassz egy megfigyelést, és magyarázd el egy csoporttársadnak. A magyarázatban különítsd el a szám értékét, a tárolható közelítést és a megjelenített szöveget.